

PRIORITY QUEUE(HEAP)

자료구조 스터디 Week10

학생회

 Made with Gamma

INDEX

01

THEORY

02

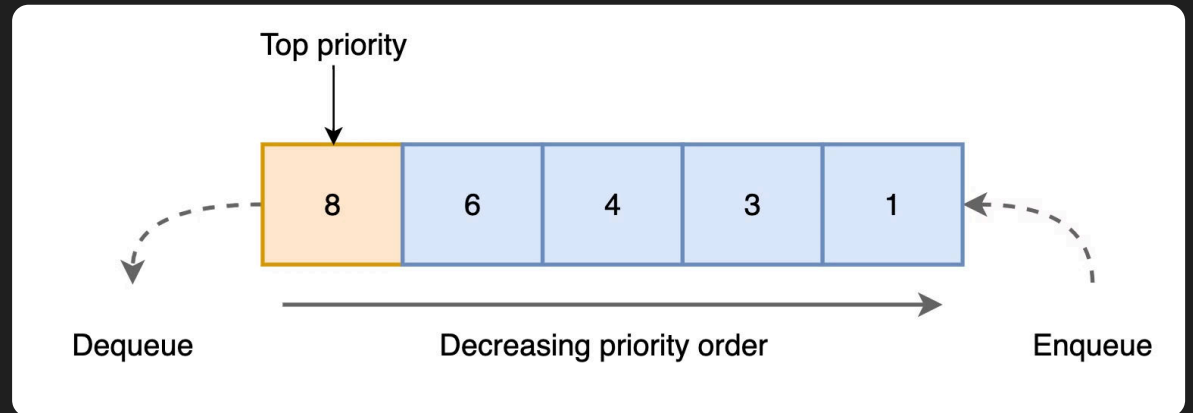
IMPLEMENTATION

03

PROBLEM SOLVING

우선순위 큐(힙)

우선순위 큐(Priority Queue)는 들어오는 순서에 상관없이 우선순위가 높은 데이터 먼저 나가는 자료구조입니다. 우선순위 큐를 이용하여 주어진 조건에 맞게 데이터를 정렬할 수 있습니다. 힙 기반 우선순위 큐는 비정렬, 정렬 시퀀스 기반 우선순위 큐보다 시간복잡도가 작습니다.



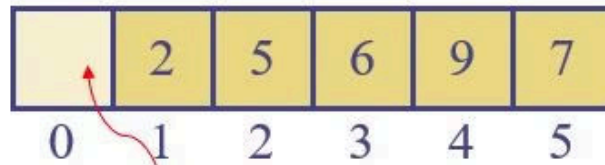
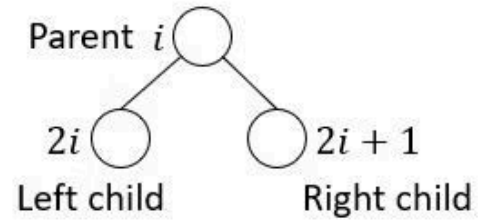
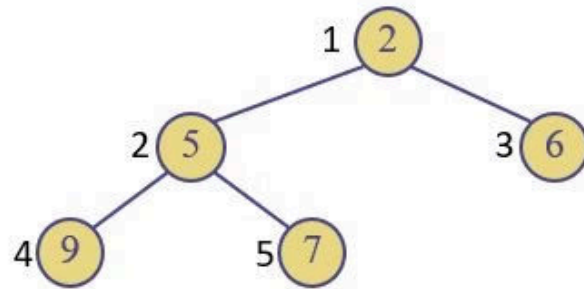
HEAP PRIORITY QUEUE



Heap

- 구조 - 트리의 마지막 레벨을 제외하고 모든 레벨의 노드가 완전히 채워져 있으며, 마지막 레벨의 모든 노드는 왼쪽부터 채워져 있는 이진 트리
- 순서 - Min Heap: 항상 parent 노드가 child 노드보다 작다
 - Max Heap: 항상 parent 노드가 child 노드보다 크다
- 구조(Complete Binary Tree), 순서(Heap-Order)

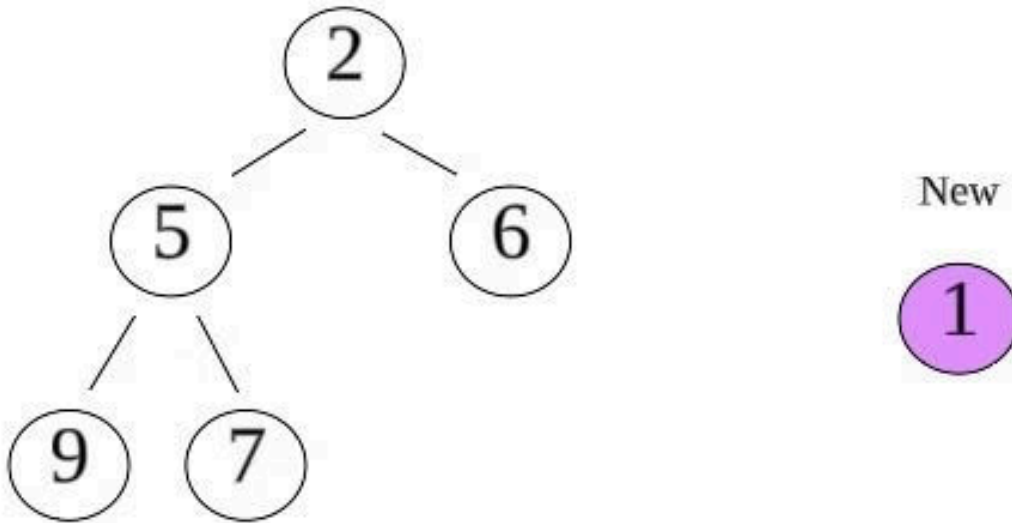
Heap



규칙성을 갖기 위해 0번 index를 비워 둔다!

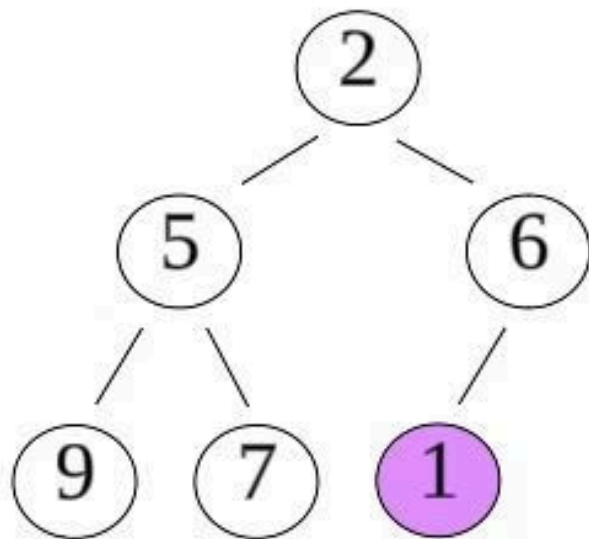
Heap - Insert

새로운 노드를 힙의 마지막 노드에 추가(구조 만족), UpHeap을 통해 순서 조정(순서 만족)



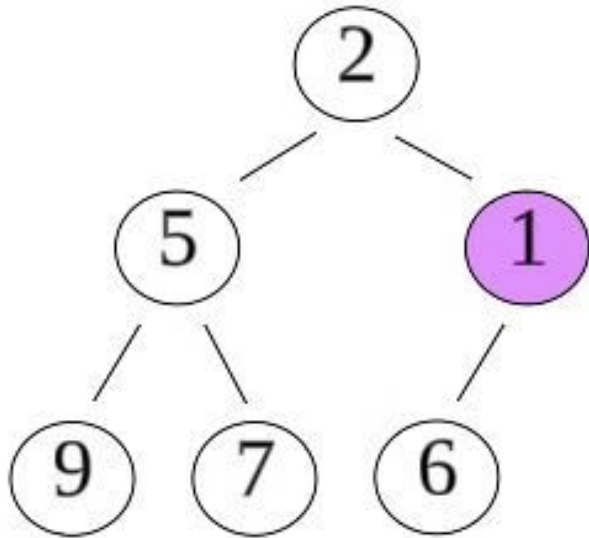
Heap - Insert

새로운 노드를 힙의 마지막 노드에 추가(구조 만족), UpHeap을 통해 순서 조정(순서 만족)



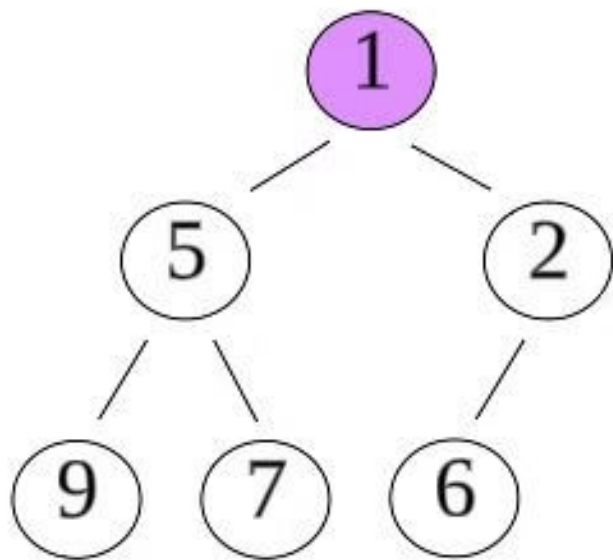
Heap - Insert

새로운 노드를 힙의 마지막 노드에 추가(구조 만족), UpHeap을 통해 순서 조정(순서 만족)



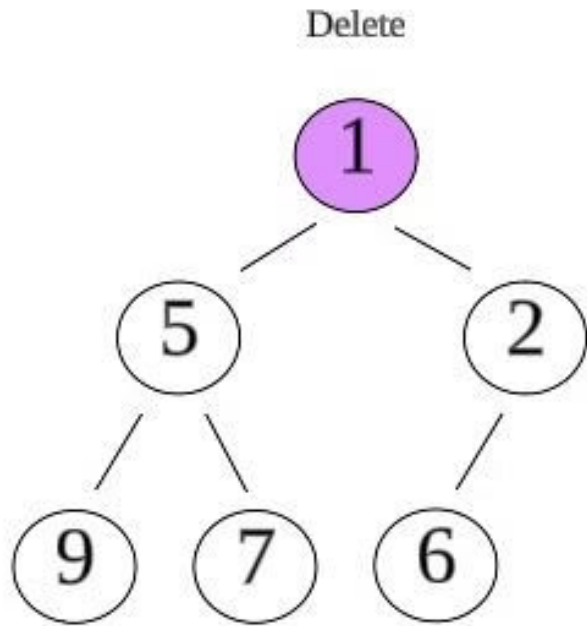
Heap - Insert

새로운 노드를 힙의 마지막 노드에 추가(구조 만족), UpHeap을 통해 순서 조정(순서 만족)



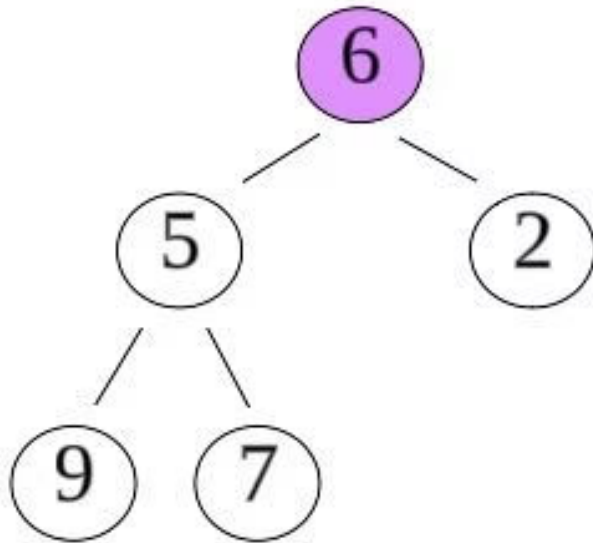
Heap - Remove

힙의 루트 삭제, 가장 마지막 노드를 루트로 이동(구조 만족), DownHeap을 통해 순서 조정(순서 만족)



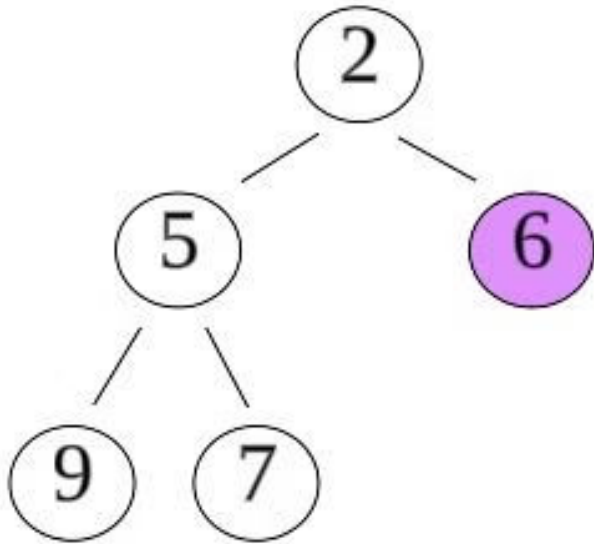
Heap - Remove

힙의 루트 삭제, 가장 마지막 노드를 루트로 이동(구조 만족), DownHeap을 통해 순서 조정(순서 만족)



Heap - Remove

힉의 루트 삭제, 가장 마지막 노드를 루트로 이동(구조 만족), DownHeap을 통해 순서 조정(순서 만족)



IMPLEMENTATION



우선순위 큐의 구현 방법

1

Heap

힙 기반 우선순위 큐

우선순위 큐의 기본 연산

1 **Insert**

삽입

2 **Remove**

삭제

멤버 변수 및 생성자

```
class Heap {  
private:  
    vector<int> arr;  
  
public:  
    Heap() {  
        arr.push_back(0);  
    }  
}
```

- arr가 Heap 역할
- 초기 상태 - idx 0에 쓰레기 값

Size & Empty

```
int size() {  
    return arr.size() - 1;  
}  
  
bool empty() {  
    return (arr.size() - 1 == 0);  
}
```

- idx 0에 쓰레기 값이 있으므로 arr의 size에서 1 뺀 값이 힙의 크기

Insert

```
void insert(int e) {  
    arr.push_back(e);  
    upHeap(size());  
}
```

- 힙의 마지막 노드에 추가(구조 만족)
- UpHeap(순서 만족)

Remove

```
void removeMin() {  
    if (empty()) {  
        cout << "-1\n";  
        return;  
    }  
  
    cout << arr[1] << '\n';  
    swap(1, size());  
    arr.pop_back();  
    downHeap(1);  
}
```

- 예외처리 및 값 출력
- 힙의 루트 삭제, 가장 마지막 노드를 루트로 이동 = 배열의 첫 원소와 마지막 원소 swap, 마지막 원소 삭제 (구조 만족)
- DownHeap(순서 만족)

Swap

```
void swap(int idx1, int idx2) {  
    arr[0] = arr[idx1];  
    arr[idx1] = arr[idx2];  
    arr[idx2] = arr[0];  
}
```

- 배열의 두 값 바꾸기
- arr[0]은 쓰레기 값이므로 temp의 역할

UpHeap

```
void upHeap(int idx) {  
    if (idx == 1) {  
        return;  
    }  
  
    int par = idx / 2;  
    if (arr[par] > arr[idx]) {  
        swap(par, idx);  
        upHeap(par);  
    }  
}
```

- $idx == 1$ ($\because$ arr[1]이 힙의 루트)라면 재귀 종료
- 현재 노드가 par보다 작다면 둘을 swap ($\because$ Min Heap-Order: 항상 부모가 자식보다 작음)
- 현재 노드 $\rightarrow$ 부모 노드 재귀

DownHeap

```
void downHeap(int idx) {
    int left = 2 * idx; int right = 2 * idx + 1;
    int child;

    if (left > size()) // 자식 존재 x
        return;

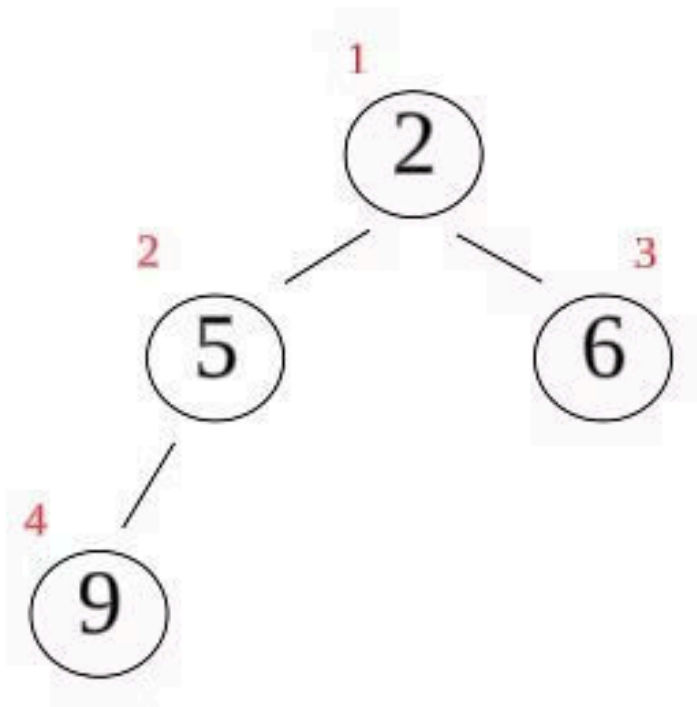
    else if (left == size()) // 왼쪽 자식만 존재
        child = left;

    else { // 두 자식 모두 존재 - 둘 비교해서 더 작은 자식 선택
        if (arr[left] <= arr[right])
            child = left;
        else
            child = right;
    }

    if (arr[child] < arr[idx]) {
        swap(child, idx);
        downHeap(child);
    }
}
```

- $\text{left} > \text{size}()$: 자식이 없는 상황이라면 재귀 종료
- $\text{left} == \text{size}()$: left만 있는 상황
 - left가 child
- else : right와 left 모두 있는 상황
 - 둘 중 작은 자식이 child
- child가 현재 노드보다 작다면 둘을 swap
($\because$ Min Heap-Order: 항상 부모가 자식보다 작음)
- 현재 노드 $\rightarrow$ 자식 노드 재귀

DownHeap



`size() : 4`

`arr.size() : 5`

PROBLEM SOLVING

**THANK'S FOR
WATCHING**